Pasqal Defining the quantum reality

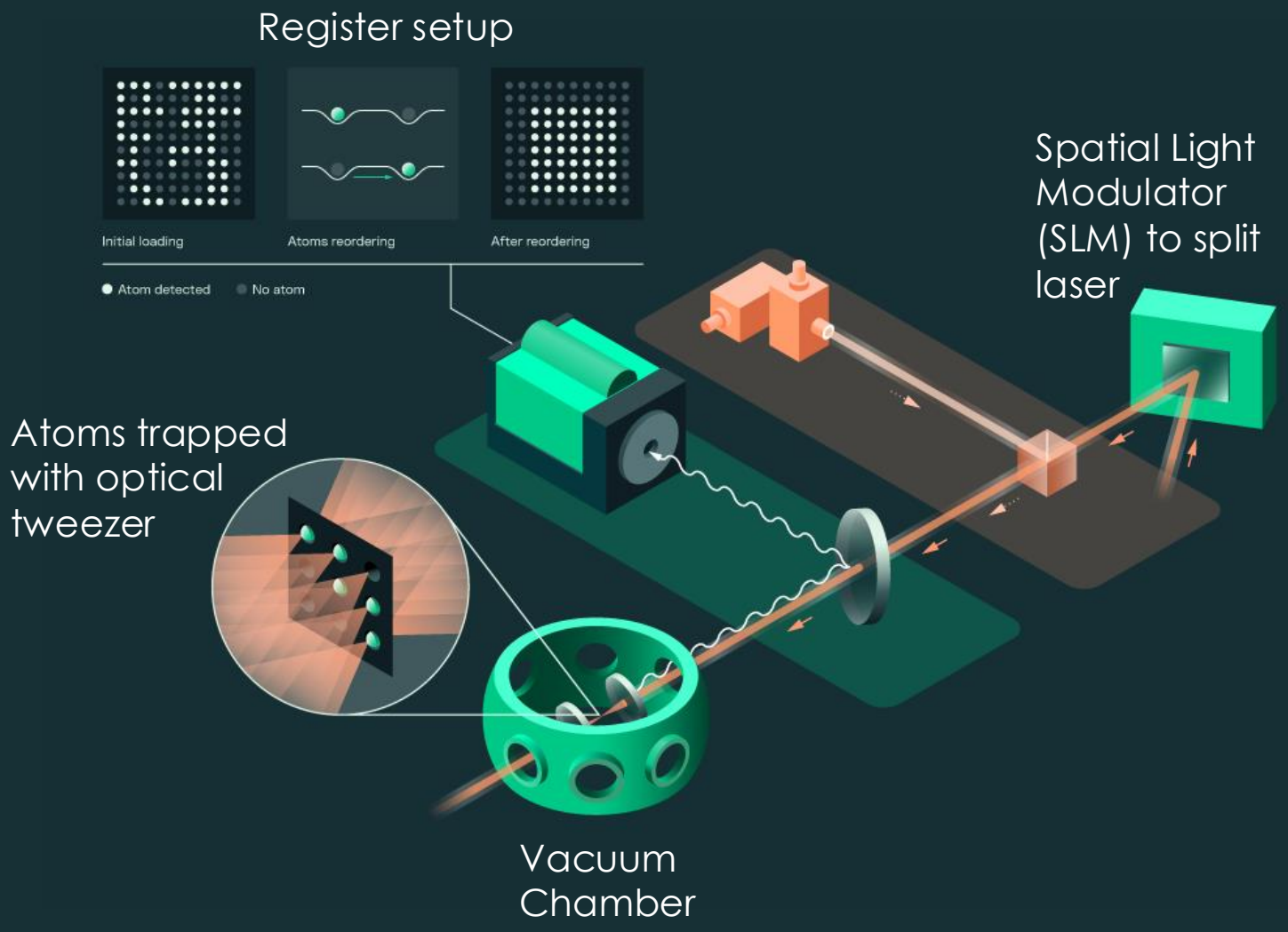
HPC-QC integration

January 19th 2025, OpenQSE

Aleksander Wennersteen
Technical lead quantum-centric HPC
Pasqal

Pasqal creates QPUs by controlling neutral atoms with lasers

Pasqal



SCALABILITY

No major roadblocks near-term to scale the qubit count to 10,000 qubits and beyond, following our roadmap

DUAL DIGITAL-ANALOG MODES

The unique dual analog-digital capability, offers the opportunity of near-term value with analog while developing Fault-Tolerant Quantum Computers

ROOM TEMPERATURE & LOW ENERGY CONSUMPTION

No cryogenics required, the system operates at room temperature, significantly reducing power consumption

First Neutral atom QPUs at datacenters

Pasqal

FRANCE HYBRID HPCQUANTUM INITIATIVE

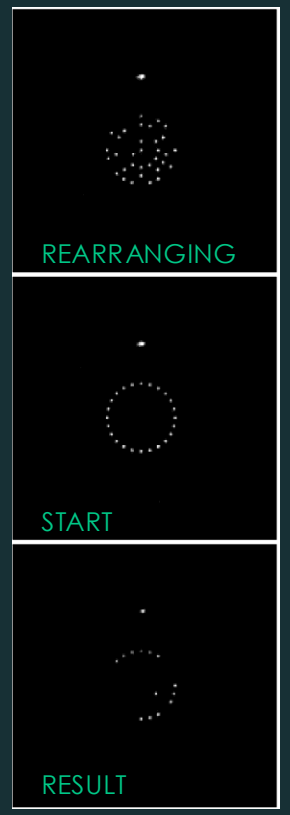
HPC|QS

EuroHPC Joint Undertaking

Orion Beta (Ruby) at CEA

GENCI

cea



First customer usecase results at Genci QPU

Orion Beta at Jülich

JÜLICH Forschungszentrum

We have successfully deployed neutral atom QPUs

Cloud QPUs (France & Canada)

25+ use cases realized with Orion Alpha in 2024-2025

Microsoft Azure

Google Cloud

Pasqal Cloud

Scaleway

OVHcloud

HPC centers

GENCI
Le calcul intensif au service de la connaissance

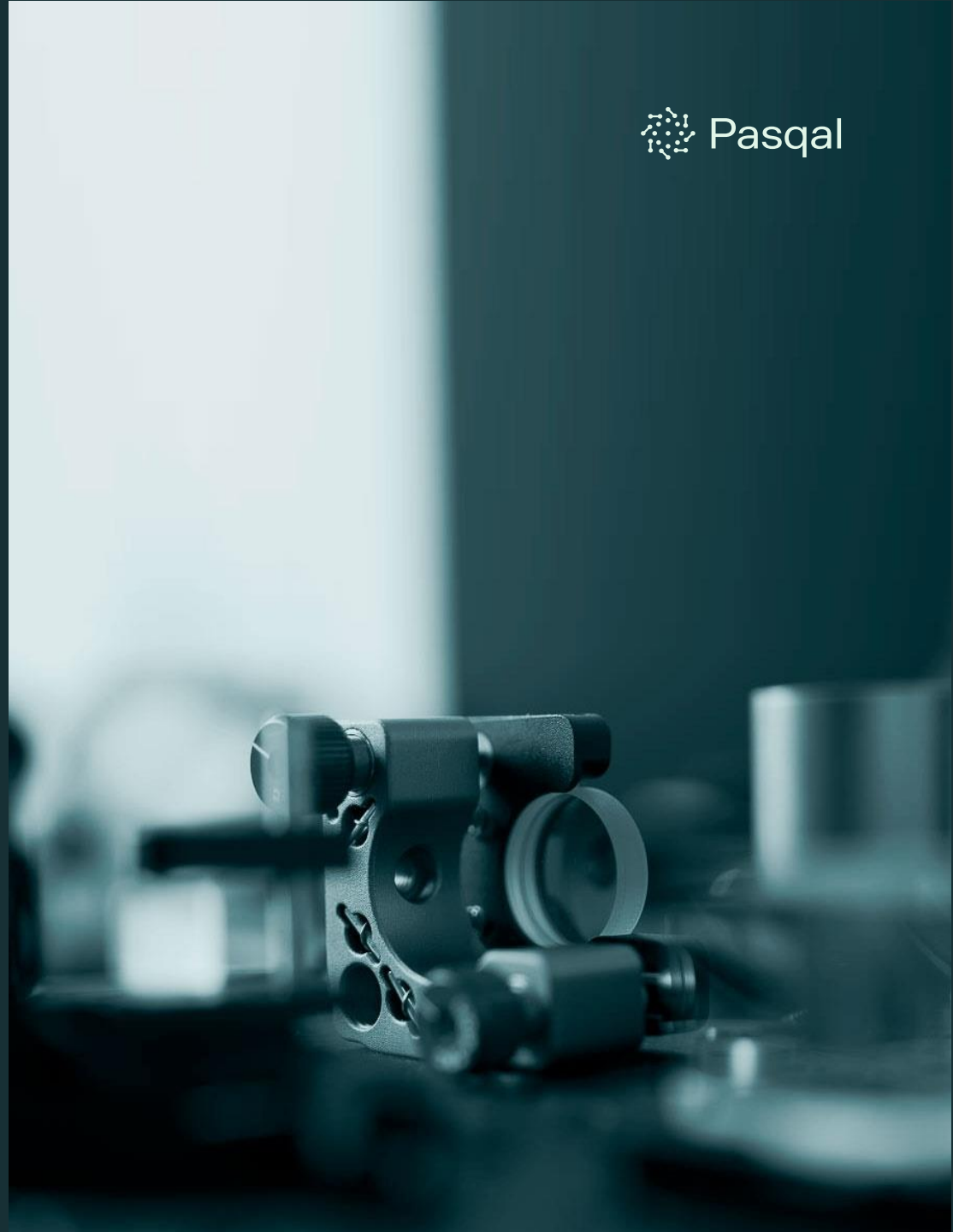
JÜLICH
Forschungszentrum

cea

Under installation

aramco

CINECA



Software – Many entry points

Pasqal

Quantum Applications

Ready-to-use quantum algorithms, designed to address specific computational challenges

MIS, QUBO (Q4 2025)
Optimization

QEK, GRIT (2026)
Graph ML

(2026)
Quantum Simulation

Quantum SDK

Create custom quantum applications with our intuitive, open-source SDK that simplifies algorithm development or use your favorite external SDK

QoolQit

Integrated external SDKs
NVIDIA CUDA-Q
IBM Qiskit
EVIDEN MyQLM

Neutral Atom Programming

Pulser (and Pulser Studio) provide a flexible ecosystem for programming neutral atom quantum processors, supporting users of all skill levels

Pulser

Execution

Access Pasqal QPUs and quantum emulators via our cloud platform, and run your jobs

HPC Middleware / Qaptiva / Pasqal Cloud Platform

Emulators on local PC

QPU

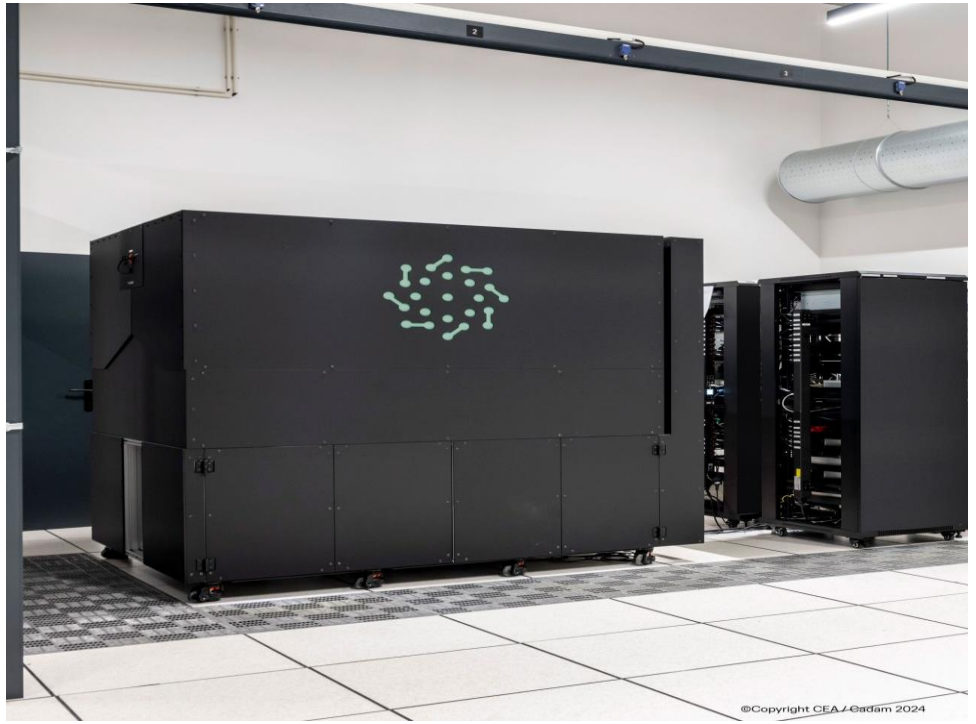
Emulators

HPC-QC integration mostly a software issue

Pasqal

What's the point of HPC-QC integration today

- User enablement (end-users, tool builders, etc…)
  - Most workloads with quantum algorithms that could provide near-term benefits are HPC based
    - Material science, quantum chemistry etc



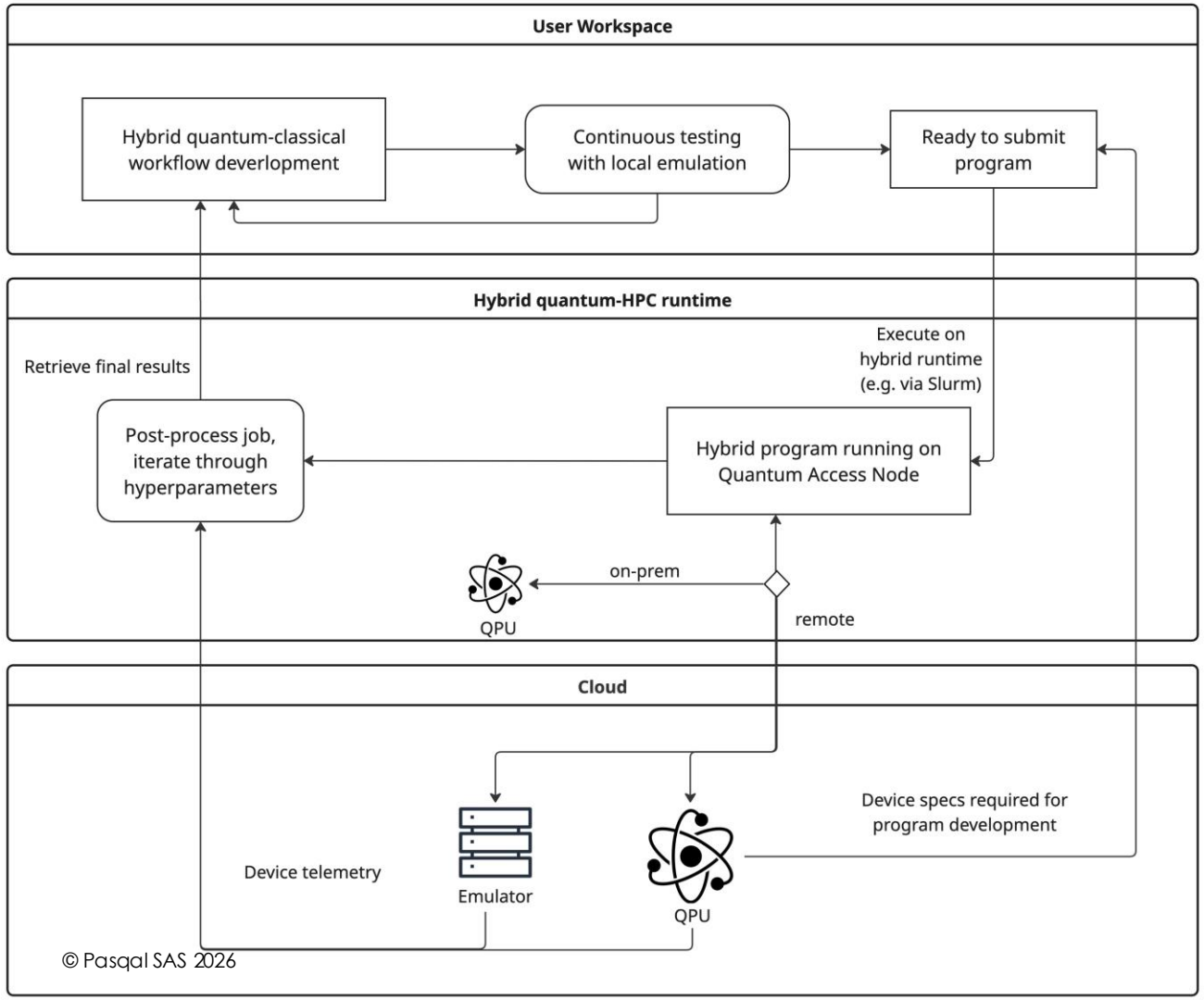
QPUs can by **physically** or **virtually** integrated in HPC environments

The software challenge

- **Scheduling, job management (Slurm, …)**
- Integratable with existing applications (C, Fortran, OpenMP, etc..)
- **Monitoring, HPC/ data center standard features (e.g. remote maintenance)**

The Developer Workflow for on-prem and cloud

Pasqal



- A full cloud backend makes it **easy to build and automate quantum workflows**.
- User-facing libraries interoperable
- Cloud and on-prem compute resources integrated in a uniform way, and **user-facing libs interoperability**
  ➔ infrastructure abstracted away from users
- Having emulators and QPUs on the same platform means **you use the same code for both**, making it simple to switch from simulation to real hardware.
  ➔ avoid env switching **reduces errors**

Diagram extended from:
A. Wennersteen et al., "Towards a user-centric HPC-QC environment", 2025, In Proc. Of the SC Workshops '25), ACM
ArXiv: 2509.20525

Pasqal

Resource management

See previous talk here by Othani and Horii Dec 15th 2025

ArXiv 2506.10052

github.com/qiskit-community/QRMI

github.com/qiskit-community/spank-plugins

Quantum resources in resource management systems

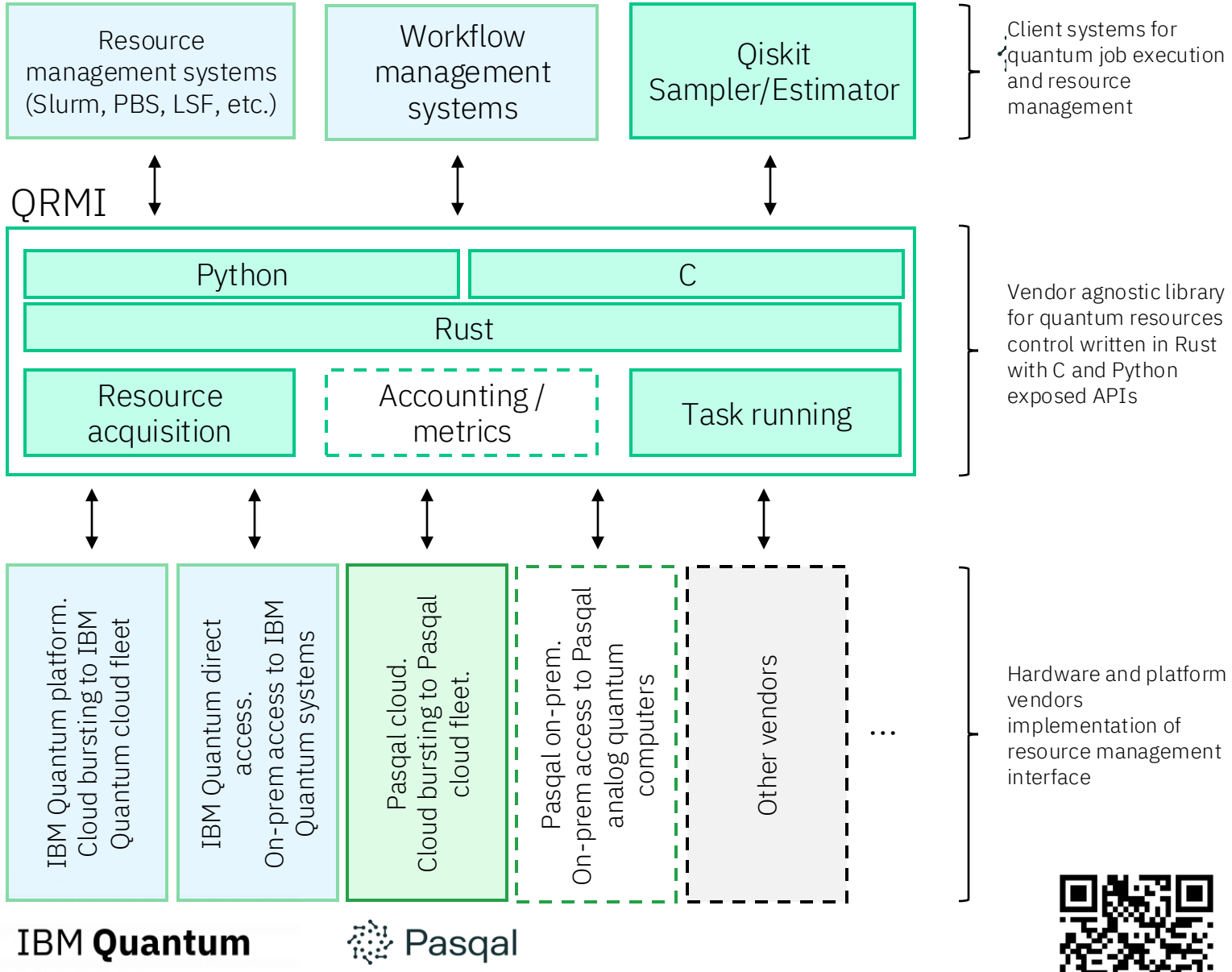
Utz Bacher,[1] Mark Birmingham,[2] Christopher D. Carothers,[3] Andrew Damin,[3] Carlos D. Gonzalez Calaza,[4,5] Ashwin Kumar Karnad,[4] Stefano Mensa,[2] Matthieu Moreau,[6] Aurelien Nober,[6] Munetaka Ohtani,[7] Max Rossmannek,[8] Philippa Rubin,[2] M. Emre Sahin,[2] Oscar Wallis,[2] Amir Shehata,[9] Iskandar Sitdikov,[10] and Aleksander Wennersteen[6]

[1] *IBM Deutschland Research & Development GmbH, Boeblingen, Germany*
[2] *The Hartree Centre, STFC, Sci-Tech Daresbury, Warrington, WA4 4AD, United Kingdom*
[3] *Center for Computational Innovation, Rensselaer Polytechnic Institute, Troy, NY USA 12180*
[4] *Jülich Supercomputing Centre, Forschungszentrum Jülich, 52425 Jülich, Germany*
[5] *RWTH Aachen University, 52056 Aachen, Germany*
[6] *PASQAL, 24 Av. Emile Baudot, 91120 Palaiseau, France*
[7] *IBM Quantum, IBM Research – Tokyo, Tokyo 103-8510, Japan*
[8] *IBM Quantum, IBM Research—Zurich, Säumerstrasse 4, 8803 Rüschlikon, Switzerland*
[9] *Oak Ridge National Laboratory, Oak Ridge, TN 37831, USA*
[10] *IBM Quantum, IBM T.J. Watson Research Center, Yorktown Heights, NY 10598, USA*

(Dated: Friday 14th November, 2025)

Quantum Resource Management Interface (QRMI)

- Thin and vendor agnostic layer to access, control and monitor underlying on-prem or cloud Quantum systems
- Exposes notion of quantum resources, which might be qubit, entire physical system, virtual QPU, etc.
- 3 sets of APIs available for QRMI:
  - Resource acquisition: availability, acquire, release.
  - Task running: execution on quantum payload on target hardware
  - [2026 1H] Accounting/metrics: system utilization, etc.
- Provides loose coupling between clients and systems



How it works

Pasqal

/etc/slurm/qrmi-config.json and plugstack.conf

```
{
  "name": "alt_marrakesh",
  "type": "qiskit-runtime-service",
  "environment": {
    "QRMI_IBM_QRS_ENDPOINT": "https://quantum.cloud.ibm.com/api/v1",
    "QRMI_IBM_QRS_IAM_ENDPOINT": "https://iam.cloud.ibm.com",
    "QRMI_IBM_QRS_IAM_APIKEY": "<YOUR IAM APIKEY FOR THIS BACKEND>",
    "QRMI_IBM_QRS_SERVICE_CRN": "<YOUR IQP INSTANCE CRN>"
  }
},
{
  "name": "FRESNEL",
  "type": "pasqal-cloud",
  "environment": {
    "QRMI_PASQAL_CLOUD_PROJECT_ID": "",
    "QRMI_PASQAL_CLOUD_AUTH_TOKEN": ""
  }
```

Sbatch script

```
#!/bin/bash

#SBATCH --job-name=pasqal_job
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH --qpu=FRESNEL

# Your script goes here
source /shared/pyenv/bin/activate
srun python /shared/spank-plugins/primitives/
```

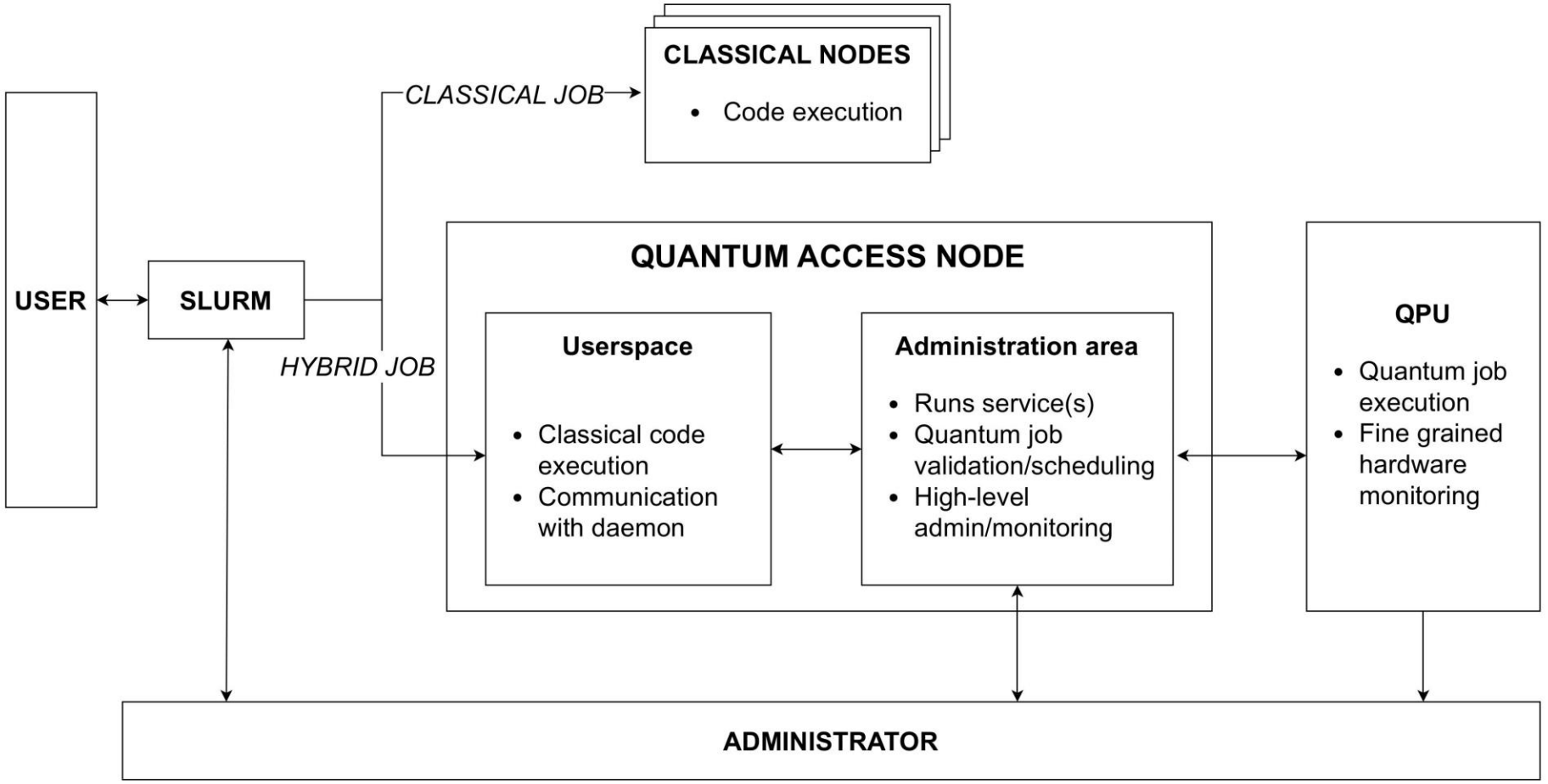
Call QRMI aware backend in code

```
from pulser_qrmi_backend.service import QRMIService
from qiskit.circuit import QuantumCircuit
from qiskit_pasqal_provider.providers.gate import HamiltonianGate
from qiskit_qrmi_primitives.pasqal.sampler import QPPSamplerV2

# Create QRMI
load_dotenv()
service = QRMIService()
resources = service.resources()
qrmi = resources[0]  # First and best QPU

...

# analog gate
gate = HamiltonianGate(ampl, det, phase, atom_coords)

# Qiskit circuit with analog gate
qc = QuantumCircuit(len(coords))
qc.append(gate, qc.qubits)

sampler = QPPSamplerV2(qrmi=qrmi)
res = sampler.run([qc])
```

Spank (and later GRES) inserts all required information as environment variables at runtime

(Or Prolog/Epilog scripts for general schedulers)

The on-prem QPU setup



A. Wennersteen et al., "*Towards a user-centric HPC-QC environment*", 2025, In Proc. Of the SC Workshops '25), ACM ArXiv: 2509.20525

2nd level scheduler

Pasqal

A second-level scheduler that mediates access to quantum resources *within* an HPC allocation.

Responsibilities

- Accept quantum job requests
- Enforce policies and priorities
- Allow admin control of QPU

Let HPC scheduler handle does well

- Node allocation
- Fair-share accounting
- Batch execution

2nd level scheduler (WIP)

Pasqal

New degrees of freedom for hybrid jobs

- Job classes
  - development, test, production
- Priority queues
- Simultaneous multi-user access
- Session persistence

Difficult to express in Slurm alone

Example policies

- Developers / test default to emulators
- Production jobs pre-empt test jobs
- Calibration aware -> block submissions

Other benefits

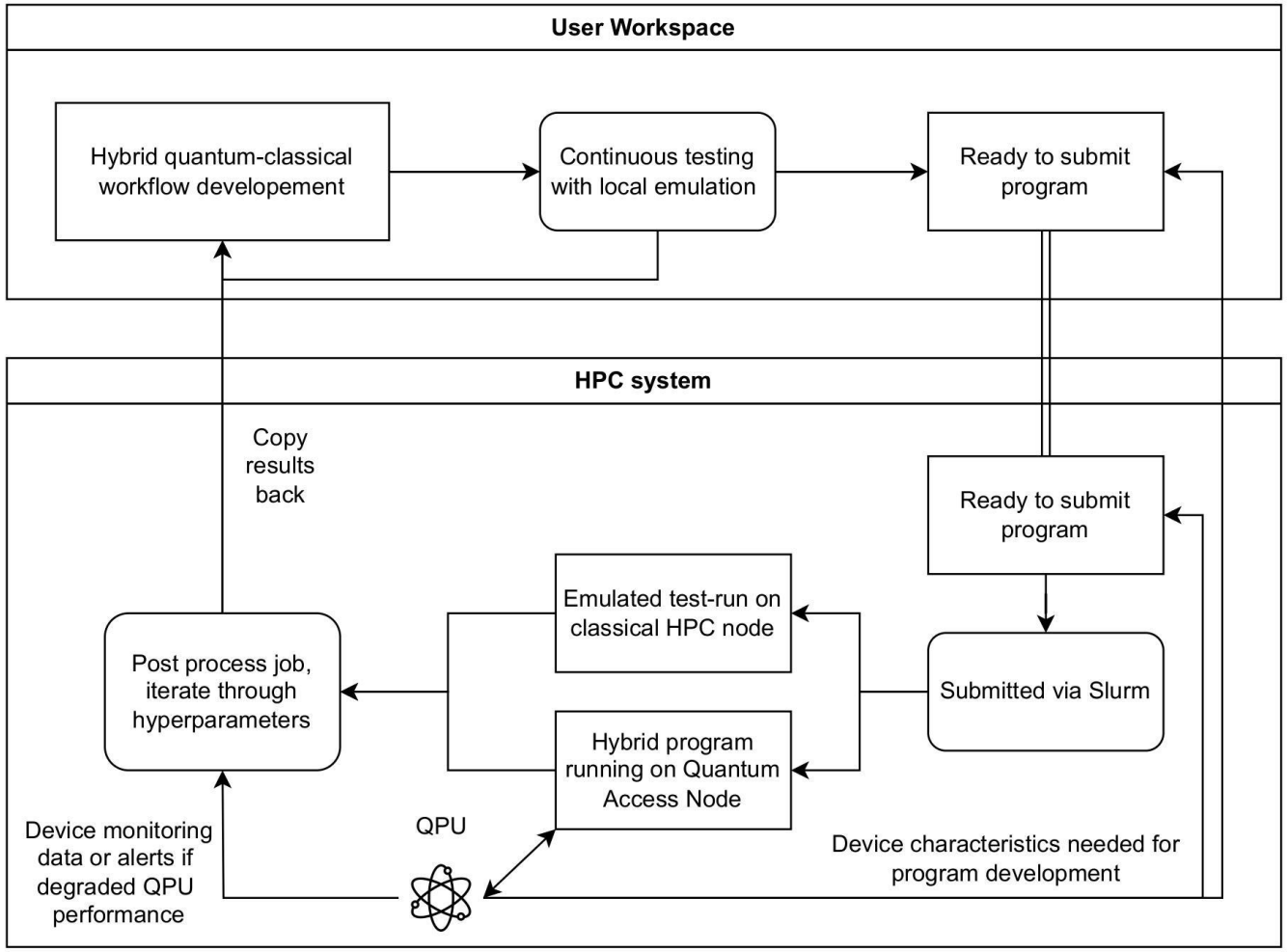
- Configurable and extendable by host
- Rapid iteration cf Slurm or embedded sw

The first step towards a more user-friendly experience

Pasqal

Development –> execution workflow

- Execution environments
  - And the myriad of SDKs
- Bugs induced by moving envs
  - *Quantum Software Engineering: Roadmap and Challenges Ahead* ACM TOCSEM 34, 5 doi.org/10.1145/371200
- Developer productivity
- Lack of ability to test quantum programs means that we'll have to take extra care of having well testable workflows



A. Wennersteen et al., "*Towards a user-centric HPC-QC environment*", 2025, In Proc. Of the SC Workshops '25), ACM ArXiv: 2509.20525

Loose ends

Pasqal

Runtime environment consistency: Provides a runtime abstraction that behaves the same way on a local developer machine as it does on a supercomputer.

Observability and monitoring: Of importance to integrate the cluster into the HPC systems existing stack, **but also:** the end-user will want/need access to certain data – this is a touchy subject. For example for profiling, debugging.

HPC-QC integration mostly a software issue

Pasqal

And this is how we address it – (summary)

- **Scheduling, job management (Slurm, …)**
  - Developing QRMI (Quantum Resource Managment Interface)
    - Solves the basic « schedule a quantum resource » problem
  - Enabling the HPC centers to perform their own research
    - With the 2nd level scheduler to be introduced
      - Open source, modifiable and customizable by customers (to some extent)
  - Shaping and adopting standards and working groups

- **Monitoring, HPC standard solutions**
  - **Monitoring**
    - Use standard solutions
    - Enable configuration and hooks
  - **HPC standards**
    - Sysadmin managable and configurable
      - Admin daemon helps us expose QPU admin features
    - Integrated « natively » with Slurm
    - 2nd level scheduler also take Slurm info like priority (tbd)

Pasqal

Thank you

Aleksander Wennersteen
Aleksander.Wennersteen@pasqal.com